

Enterprise Features & Architecture Operations Manual

A Comprehensive Reference to Dual-Plane Master & Satellite Taxonomy, UI Placements, Operator Workflows, and Technical Background Lifecycles

Target System: Azam-Pnet / PNETLab Enterprise	Document Version: 2.5.0-LTS Enterprise Edition
Generated: September 19, 2026	Architectural Planes: Master Control Plane + Satellite Compute Plane
System Daemons: azambasha-ops-api (:8088), azam-watchdog.service	Security & Role Guard: RBAC Admin Guard (Role 0) + Airgap Packaging

1. Executive Architectural Taxonomy & Feature Placements

Azam-Pnet organizes all twenty-five enterprise capabilities across three deliberate user and runtime environments: **Outside-the-Lab Canvas** (global hypervisor diagnostics, template repository marketplace, multi-cloud and offline air-gapped backups, capacity planning, and SSL automation accessed from the main administrative dashboard), **Inside-the-Lab Canvas** (in-workbench workflow tools, live traffic bandwidth heatmaps, OS-level desktop push notifications, anti-bootstorm staggered startup with KSM deduplication, Git version control, and live Wireshark packet capture available right inside the active topology view), and **Autonomous Kernel & Background Daemons** (24/7 self-healing watchdogs, Soft-RoCE RXE MTU 9000 kernel engines, MySQL socket auto-healers, and nightly SSD TRIM cron jobs running continuously in the host background).

Feature Name	Architectural Zone	Exact UI / System Location	Primary Trigger	Plane Support
1. Community Templates Marketplace	Outside Lab	Main Nav > Azam Features > Community Templates	One-Click Import	Master Only
2. Live Hot-Node Profiler (RBAC Guard)	Outside Lab	Main Nav > Azam Features > Health & Diagnostics	Real-Time / Kill	Master & CLI
3. HTML5 Console Session Reset	Outside Lab	Main Nav > Azam Features > Console Fixer	Repair Button	Master Only
4. Local, Cloud & Air-Gapped Backups	Outside Lab	Main Nav > Azam Features > Cloud & Local Backup	Instant Archive	Master & CLI
5. Fleet Multi-Node Cluster Monitor	Outside Lab	Main Nav > Azam Features > Cluster Nodes	Auto-Refresh (15s)	Master (Agent: Sat)
6. Lab Hardware Capacity Planner	Outside Lab	Main Nav > Azam Features > Capacity Planner	Interactive Sizer	Master Only
7. Comprehensive Diagnostic Doctor	Outside Lab	Main Nav > Azam Features > System Doctor	Run Diagnostics	Master & Satellite
8. QCOW2 Disk Shrinker & Sparse Comp.	Outside Lab	Main Nav > Azam Features > Storage Optimizer	Batch Compress	Master & Satellite
9. Multi-Cloud Transit Overlay Bridge	Outside Lab	Main Nav > Azam Features > Cloud Transit	Connect Overlay	Master & Satellite
10. Automated SSL & WhatsApp Alerts	Outside Lab	Main Nav > Azam Features > Alerts & SSL	Deploy Cert / Test	Master Only
11. Version Synchronizer & PDF Portal	Outside Lab	Main Nav > Azam Features > Updates & Header Doc	Check & Update	Master & Satellite
12. Anti-Bootstorm & In-Canvas KSM	Inside Canvas	Canvas Workbench > Top Toolbar > Anti-Bootstorm	Toolbar Button	Master (KSM: Both)

13. Topology Git VCS & Auto-Commit	Inside Canvas	Canvas Left Sidebar > Git VCS Icon > Drawer	Auto / Commit	Master Only
14. In-Workbench Quick Console Healer	Inside Canvas	Canvas Workbench > Top Toolbar > Fix Console	Toolbar Button	Master Only
15. Live Wireshark & Traffic Heatmap	Inside Canvas	Top Toolbar > Heatmap / Right-Click > Capture	Toolbar & Menu	Master & Satellite
16. AI Lab Copilot & Desktop Alerts	Inside Canvas	Canvas Bottom-Right > Floating AI & OS Notif	Chat & Event Hook	Master Only
17. Multi-Node Config Diff & Rollback	Inside Canvas	Canvas Workbench > Management Tools > Config Diff	Compare Configs	Master Only
18. Automated Lab Exam Grader	Inside Canvas	Canvas Workbench > Lab Actions > Run Evaluation	Evaluate Lab	Master Only
19. Ping Mesh & Traffic Generator	Inside Canvas	Canvas Workbench > Lab Testing > Traffic Injector	Generate Traffic	Master & Satellite
20. High-Res Diagram & SVG Exporter	Inside Canvas	Canvas Workbench > Export > Vector Topology	Download SVG	Master Only
21. Interactive Canvas Accelerators	Inside Canvas	Canvas Navigation Controls (Minimap & Snap)	Always Active	Master Only
22. 24/7 Self-Healing Watchdog	Daemon Plane	systemd: azam-watchdog.service	Autonomous (60s)	Master & Satellite
23. MySQL Socket & Credential Healer	Daemon Plane	systemd & Cron: azambasha-fix-web-credentials	On Boot & Hourly	Master Only
24. Soft-RoCE RXE & Jumbo MTU Engine	Kernel Plane	Kernel Module (ib_core, rdma_rxe) & udev	Boot & Net Hook	Master & Satellite
25. Scheduled Maintenance & TRIM Cron	Daemon Plane	cron.d: azambasha-maintenance-trim	Daily 03:00 UTC	Master & Satellite

2. Master vs. Satellite Dual-Plane Architecture & Alignment Matrix

A foundational design principle of the Azam-Pnet Enterprise distribution is strict **architectural role separation** between the **Master Management Node** (the control plane) and **Satellite Compute Nodes** (the data and execution plane). Understanding this dual plane is critical for multi-hypervisor cluster deployments.

1. Master Node Architectural Role (Control & Presentation Plane):

The Master Node hosts the central user interface (Apache2/Nginx), the MariaDB database engine, the Guacamole HTML5 console proxy (guacd and tomcat9), the central REST API daemon (azambasha-ops-api on port 8088), the Git topology repository root, and all interactive canvas workbench tools. Administrators and lab students log into the Master Node URL. When a heavy multi-vendor lab topology is launched across a cluster, the Master Node acts as the orchestrator, assigning virtual machine workloads to Satellite compute nodes according to real-time RAM and CPU availability.

2. Satellite Node Architectural Role (Headless High-Performance Compute Plane):

Satellite nodes are dedicated, bare-metal or nested KVM execution engines. Their primary responsibility is running heavy virtual machines (e.g., Cisco IOS-XRv9k, Arista cEOS, Junos vMX, Linux appliances) with zero CPU cycles wasted on unnecessary web servers, SQL databases, or desktop environments. Therefore, Satellite nodes operate in a **headless compute state**. Running azambasha-install-azam-features.sh --satellite on a satellite node purposefully provisions all 26 CLI system tools, installs the 24/7 watchdog daemon (azam-watchdog.service), tunes Kernel Samepage Merging (KSM), loads Soft-RoCE RXE acceleration with MTU 9000 jumbo frames, and installs the nightly SSD TRIM maintenance cron, while omitting Apache, Guacamole, and MariaDB.

Platform Component / Feature	Master Node Execution	Satellite Node Execution	Parity & Architectural Rationale
Web GUI & Canvas Workbench	Active (Apache2 / PHP / HTML5)	Disabled (Zero Web Overhead)	Aligned: Satellites are headless compute workers; users access central Master GUI.
azambasha-ops-api (:8088)	Active (Central REST Engine)	Active or Cluster-Bridged	Aligned: Master serves endpoints; Satellites expose node telemetry via cluster hooks.
24/7 System Watchdog	Active (azam-watchdog.service)	Active (azam-watchdog.service)	100% Identical: Watchdog self-heals KVM, bridges, and processes on both planes.
Enterprise CLI Suite (26 Tools)	Linked in /usr/local/bin/	Linked in /usr/local/bin/	100% Identical: Administrators have identical CLI commands on every host.
Kernel Samepage Merging (KSM)	Enabled & Tuned	Enabled & Aggressively Tuned	100% Identical: Maximizes memory deduplication across identical appliances.
Soft-RoCE RXE & MTU 9000	Loaded (rdma_rxe, ib_core)	Loaded (rdma_rxe, ib_core)	100% Identical: Unlocks wire-speed inter-node packet fabric without fragmentation.
SSD TRIM & Log Maintenance	Scheduled (Nightly 03:00 UTC)	Scheduled (Nightly 03:00 UTC)	100% Identical: Reclaims deleted flash blocks and purges stale sockets.
MariaDB & Guacamole Socket Fixer	Active (Database on Master)	Not Applicable (Headless)	Aligned: Satellites do not host MySQL or Guacamole; no repair needed.
Air-Gapped Pack Bundler	Full Pack Generator (GUI & CLI)	CLI Extractor / Consumer	Aligned: Master builds the tarball; Satellites consume bundle for air-gapped install.

3. Outside-the-Lab Features (Main Dashboard: /main/#/azam-features)

Outside-the-lab features are located in the primary management dashboard (/main/#/azam-features) accessible prior to loading any topology. They provide centralized hypervisor monitoring, repository synchronizations, fleet health oversight, air-gapped bundling, RBAC access control, and disaster recovery.

1. Community Templates Marketplace [GUI: Outside Lab Canvas]

UI & System Placement: Main Dashboard Navigation > Azam Features > Community Templates tab (#pane-templates)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Navigate to the top administrative bar and click **Azam Features** (or open URL /main/#/azam-features).
- **Step 2:** Select the **Community Templates** tab located in the left navigation sidebar.
- **Step 3:** Browse or filter the community catalog (Cisco, Arista, Juniper, Fortinet, Linux, MikroTik).
- **Step 4:** Click the **Import Template** button next to the desired network operating system template.
- **Step 5:** Observe the live progress bar as image schemas and YAML definitions are verified.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Front-end sends HTTP GET /api/azam/templates to azambasha-ops-api (port 8088).
- **Step 2:** API server reads /opt/unetlab/html/templates/ and parses device definitions.
- **Step 3:** When import is clicked, API issues POST /api/azam/templates/import with template ID.
- **Step 4:** Python backend queries repository manifest schemas or git repository.
- **Step 5:** Verifies template file syntax, generates corresponding /opt/unetlab/html/templates/{os}.yml, sets permissions to www-data:www-data 0644, and flushes APC/OpCache.

2. Live Hot-Node Profiler & Task Manager (RBAC Guard) [GUI: Outside Lab Canvas]

UI & System Placement: Main Dashboard Navigation > Azam Features > Health & Diagnostics (#pane-health)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Navigate to /main/#/azam-features and select the **Health & Diagnostics** tab.
- **Step 2:** Scroll down to the **Hot-Node Live Performance Table**.
- **Step 3:** Inspect real-time consumption showing PID, Node Name, Lab Name, CPU %, and Memory %.
- **Step 4:** If a node is looping or hung, click the red **Kill Process** button in the Action column.
- **Step 5:** If logged in as non-admin, the system enforces RBAC: process termination is disabled and locked to Admin role 0 only.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Front-end polls GET /api/azam/perf/top every 4 seconds.
- **Step 2:** Backend runs ps aux --sort=-%cpu, filtering for QEMU (qemu-system-x86_64), IOL, and Dynamiqs.
- **Step 3:** Correlates PID to Lab UUID and Node ID via /opt/unetlab/tmp/ sockets.
- **Step 4:** On Kill trigger, POST /azam-ops/api/perf-kill inspects the X-User-Role header; if not role 0 (Admin), returns HTTP 403 Forbidden.
- **Step 5:** For authorized admins, sends SIGTERM (15) to PID; if unresponsive after 2.5s, sends SIGKILL (9) and logs to /var/log/azam-ops-api.log.

3. HTML5 Console Auto-Fixer & Session Cleaner [GUI: Outside Lab Canvas]

UI & System Placement: Main Dashboard Navigation > Azam Features > Console Fixer (#pane-console)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** In the main dashboard, go to **Azam Features > Console Fixer**.
- **Step 2:** Click the primary button labeled **Run Console Full Repair**.

- **Step 3:** Wait 3-5 seconds while the system executes the recovery sequence.
- **Step 4:** Review the output modal confirming Guacamole, Tomcat9, and Guacd service reloads.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** API endpoint POST /api/azam/console-fix-full triggers azambasha-fix-web-credentials.sh --fix-guac.
- **Step 2:** System script kills orphaned websockify and stale Guacamole daemon worker threads.
- **Step 3:** Checks /etc/guacamole/guacamole.properties and MariaDB Guacamole user authorization tables.
- **Step 4:** Executes systemctl restart guacd tomcat9 and verifies local socket listening on 127.0.0.1:4822.
- **Step 5:** Flushes browser websocket cookies and returns JSON status {success: true, services_reloaded: 3}.

4. Local, Multi-Cloud & 100% Air-Gapped Bundler [\[GUI: Outside Lab Canvas\]](#)

UI & System Placement: Main Dashboard Navigation > Azam Features > Cloud & Local Backups (#pane-cloud)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Navigate to **Azam Features > Cloud & Local Backups** tab.
- **Step 2:** To create an immediate local archive, click **Create Local Archive Now**.
- **Step 3:** To sync to remote cloud storage, configure AWS S3, Google Cloud, or Mega credentials in settings and click **Sync to Cloud**.
- **Step 4:** For isolated high-security environments, click **Generate Offline Airgap Bundle (.tar.gz)** in the Airgap Packaging card.
- **Step 5:** Once generated, click the direct download link or retrieve the tarball from /opt/unetlab/data/Exports/.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Request POST /azam-ops/api/airgap-pack or POST /api/azam/backups/local dispatches the bundler.
- **Step 2:** For Airgap packs, azambasha-airgap-pack.sh creates an offline archive containing Debian package archives, pip wheels, schemas, scripts, and web UI components.
- **Step 3:** Moves generated tarball to /opt/unetlab/data/Exports/azam-pnet-airgap-bundle-latest.tar.gz with read permissions.
- **Step 4:** For cloud backups, azambasha-cloud-backup.sh executes rclone sync with AES-256 encryption and validates SHA-256 checksums.

5. Fleet Multi-Node Cluster Monitor [\[GUI: Outside Lab Canvas\]](#)

UI & System Placement: Main Dashboard Navigation > Azam Features > Cluster Fleet (#pane-cluster)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Go to **Azam Features > Cluster Fleet Monitor** tab.
- **Step 2:** View all connected PNETLab satellite worker nodes, CPU/RAM utilization, and cluster health.
- **Step 3:** Click **Add Satellite Node** and enter worker IP, SSH Port, and Shared Token.
- **Step 4:** Click **Balance Workload** to redistribute virtual nodes across cluster hypervisors.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Backend daemon azambasha-ops-api queries satellite worker endpoints every 15s via HTTPS/mTLS.
- **Step 2:** Aggregates remote CPU, RAM, and bridge interface states into centralized memory cache.
- **Step 3:** Evaluates node placement affinity rules against worker RAM capacity prior to new VM dispatch.
- **Step 4:** If a worker becomes unresponsive (missing 3 heartbeats), marks node as Degraded and triggers alert hook.

6. Lab Capacity & Hardware Sizing Planner [\[GUI: Outside Lab Canvas\]](#)

UI & System Placement: Main Dashboard Navigation > Azam Features > Capacity Planner (#pane-capacity)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Open **Azam Features > Capacity & Sizing Planner** tab.
- **Step 2:** Enter estimated count of heavy appliances (e.g., Cisco XRv9k, Arista cEOS, Junos VMX, Linux servers).
- **Step 3:** The interactive calculator displays required vCPU, RAM, QCOW2 storage, and recommended KVM overcommit ratios.
- **Step 4:** Click **Apply Resource Limits** to enforce memory guardrails.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Calculator computes total theoretical footprint against actual available host hardware queried via `/proc/meminfo`.
- **Step 2:** Checks KSM (Kernel Samepage Merging) deduplication metrics via `/sys/kernel/mm/ksm/pages_sharing`.
- **Step 3:** Enforces safe limits by writing updated swap thresholds into `/etc/sysctl.d/99-pnetlab-perf.conf`.
- **Step 4:** Notifies administrator if requested deployment exceeds safe memory thresholds (over 85% host RAM).

7. Comprehensive Diagnostic Doctor [\[GUI: Outside Lab Canvas\]](#)

UI & System Placement: *Main Dashboard Navigation > Azam Features > System Doctor (#pane-doctor)*

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Navigate to **Azam Features > System Doctor** tab.
- **Step 2:** Click **Run Full System Diagnostic**.
- **Step 3:** Observe 25+ automated checks spanning MySQL sockets, disk inodes, KVM permissions, bridge MTU, and DNS.
- **Step 4:** Click **Auto-Heal Detected Issues** to execute automated remediation.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Triggers execution of `/usr/local/bin/azambasha-doctor`.
- **Step 2:** Inspects MariaDB status via unix socket `/var/run/mysqld/mysqld.sock`.
- **Step 3:** Checks `/dev/kvm` permissions (0666), IP forwarding (`net.ipv4.ip_forward=1`), and bridge states.
- **Step 4:** If auto-heal is selected, executes repair routines, resets permissions on `/opt/unetlab/`, and returns diagnostic report.

8. QCOW2 Disk Shrinker & Sparse Compressor [\[GUI: Outside Lab Canvas\]](#)

UI & System Placement: *Main Dashboard Navigation > Azam Features > Storage Optimizer (#pane-storage)*

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Navigate to **Azam Features > Storage Optimizer** tab.
- **Step 2:** Select target lab or device image directory from the dropdown menu.
- **Step 3:** Click **Scan for Reclaimable Space**.
- **Step 4:** Click **Shrink & Compress Selected Images** and observe real-time reclaimed gigabytes.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Background worker scans `/opt/unetlab/addons/qemu/` and active lab storage.
- **Step 2:** Executes `qemu-img info` to calculate virtual size vs. actual allocated disk blocks.
- **Step 3:** Invokes `azambasha-heavy-node-optimizer.sh --shrink` which runs `qemu-img convert -c -O qcow2` with zlib compression.
- **Step 4:** Replaces original image atomically, updating sparse metadata and freeing unused hypervisor blocks.

9. Multi-Cloud Transit Overlay Bridge [\[GUI: Outside Lab Canvas\]](#)

UI & System Placement: *Main Dashboard Navigation > Azam Features > Cloud Transit (#pane-transit)*

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Open **Azam Features > Cloud Transit** tab.
- **Step 2:** Select transit provider: **Tailscale**, **ZeroTier**, or **WireGuard**.
- **Step 3:** Paste network auth key or configuration token and click **Connect Transit Bridge**.
- **Step 4:** Interconnect local lab Management Cloud (pnet0/pnet1) directly to AWS VPC, Azure VNet, or home LAN.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** API executes `azambasha-cloud-transit.sh` with selected provider parameters.
- **Step 2:** Brings up virtual tun/tap interface (e.g. `tailscale0` or `wg0`) in host kernel.
- **Step 3:** Configures iptables `-t nat -A POSTROUTING -o <tun_interface> -j MASQUERADE` and enables proxy ARP.
- **Step 4:** Bridges virtual lab nodes directly to the remote cloud subnet without requiring public IPs.

10. Automated SSL & WhatsApp Alerts [GUI: Outside Lab Canvas]

UI & System Placement: Main Dashboard Navigation > **Azam Features** > **Alerts & SSL** (#pane-alerts)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Go to **Azam Features** > **Alerts & SSL** tab.
- **Step 2:** For SSL: enter fully qualified domain name (FQDN) and admin email, then click **Deploy Let's Encrypt SSL**.
- **Step 3:** For Alerts: enter Twilio Account SID, Auth Token, and WhatsApp recipient number.
- **Step 4:** Click **Send Test WhatsApp Alert** to verify real-time notification delivery.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** SSL process runs certbot certonly --standalone -d domain.com, binding temporarily to port 80.
- **Step 2:** Automatically updates Nginx configuration in /etc/nginx/sites-available/default with TLS certificates and HTTP/2.
- **Step 3:** Reloads Nginx and sets up automatic certbot renewal cron job in /etc/cron.d/certbot.
- **Step 4:** WhatsApp service stores encrypted credentials in /etc/pnetlab/alerts.conf and transmits JSON payload via Twilio REST API.

11. Version Synchronizer & Direct Operations Manual Portal [GUI: Outside Lab Canvas]

UI & System Placement: Main Dashboard Header > **Operations Manual (PDF) & Updates Tab** (#pane-updates)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** In the top navigation bar of the main dashboard, click the **Operations Manual (PDF)** button.
- **Step 2:** The document opens directly in a new browser tab with high-resolution vector layout.
- **Step 3:** To update the platform, go to **Azam Features** > **Updates & Engine** tab.
- **Step 4:** Click **Check for Feature Updates** to compare local git commit against upstream release.
- **Step 5:** Click **Apply Updates & Re-link CLI Tools** to refresh system binaries.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Top header button directs browser to /azam-ops/api/docs/manual or /docs/manual.pdf.
- **Step 2:** REST API serves PDF directly from /opt/azambasha/docs/ with Content-Type: application/pdf.
- **Step 3:** When update is clicked, queries git repository commit HEAD.
- **Step 4:** Executes azambasha-install-azam-features.sh, updating CLI symlinks in /usr/local/bin/ and restarting azambasha-ops-api.service.

4. Inside-the-Lab Canvas Features (Lab Workbench: /themes/default/)

Inside-the-lab features are integrated directly into the visual network topology canvas. They empower network engineers to execute live troubleshooting, staggered boot scheduling with KSM deduplication, automated version control commits, live traffic heatmaps, packet sniffing, and AI copilot queries without leaving the workbench.

12. Anti-Bootstorm Staggered Startup & In-Canvas KSM Booster [GUI: Inside Lab Canvas]

UI & System Placement: Lab Workbench Top Toolbar > **Anti-Bootstorm** button (#btn-azam-bootstorm)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Open any lab topology inside the PNETLab canvas workspace.
- **Step 2:** In the top workbench navigation toolbar, locate the purple button labeled **Anti-Bootstorm** (icon: lightning bolt).
- **Step 3:** In the modal, check the box **Boost with KSM Memory Deduplication**.
- **Step 4:** Select startup profile: **Conservative (15s delay)**, **Balanced (10s delay)**, or **Aggressive (5s delay)**.
- **Step 5:** Click **Start All Nodes Safely** and watch nodes boot sequentially without hypervisor overload.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Trigger executes `azamLaunchBootstormModal()` which fetches node list and KSM status.
- **Step 2:** If KSM booster is checked, client dispatches POST `/azam-ops/api/node-ksm-tune` setting `/sys/kernel/mm/ksm/run=1`.
- **Step 3:** Backend runs `azambasha-bootstorm.py` ordering nodes by weight (firewalls/spines first).
- **Step 4:** Inspects CPU utilization via `/proc/stat`; if load exceeds 80%, pauses until CPU stabilizes.
- **Step 5:** Boots nodes individually via `unl_wrapper -a start -i {id}`, preventing hypervisor OOM panics.

13. Topology Git Version Control & Automated Auto-Commit [GUI: Inside Lab Canvas]

UI & System Placement: Lab Workbench Left Sidebar > **Git VCS** icon (#btn-azam-git-vcs) > Slide-Over Drawer

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Inside the lab canvas, click the **Git Version Control** icon in the left toolbar (branch icon, #btn-azam-git-vcs).
- **Step 2:** A slide-over drawer displays commit history, authors, and timestamps.
- **Step 3:** When you add nodes, connect interfaces, or save the lab, the system triggers an **Automated Auto-Commit** in the background.
- **Step 4:** To create a named checkpoint: enter a commit message (e.g. 'Configured OSPF Area 0') and click **Commit Topology State**.
- **Step 5:** To rollback: click **Restore** next to any historical commit; the canvas automatically reloads.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** On drawer open, fetches commit history via GET `/api/azam/topology-log?lab={lab_name}`.
- **Step 2:** Ensures lab folder `/opt/unetlab/labs/<lab>` is initialized with git; if not, runs `git init`.
- **Step 3:** When topology changes occur, front-end issues POST `/azam-ops/api/topology-autocommit`.
- **Step 4:** Stages the `.unl XML` file and creates a git commit with timestamp and change description.
- **Step 5:** On rollback, checks out historical snapshot via `git checkout <hash> -- <lab.unl>` and dispatches reload event.

14. In-Workbench Quick Console Healer [GUI: Inside Lab Canvas]

UI & System Placement: Lab Workbench Top Toolbar > **Fix Console** button (#btn-azam-fix-console)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** While inside a lab, if clicking a node yields a blank screen or 'Connection Refused', stay on the canvas.
- **Step 2:** Click the green button in the top toolbar labeled **Fix Console** (icon: wrench).
- **Step 3:** A confirmation toast appears: 'Restarting HTML5 Console Services...'
- **Step 4:** Wait 2 seconds until the success notification appears ('Console services recovered successfully').

- **Step 5:** Re-click the node to open the working HTML5 terminal immediately.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Button executes `azamFixConsoleNow()` sending asynchronous POST `/api/azam/console-fix-full`.
- **Step 2:** API server invokes `azambasha-fix-web-credentials.sh` targeting Guacamole and Tomcat daemon sockets.
- **Step 3:** Flushes stuck client sessions in Guacamole memory cache and verifies port 4822 binds properly.
- **Step 4:** Resets browser terminal session iframe tokens without requiring a full page reload.

15. Live Wireshark Sniffer & In-Canvas Traffic Heatmap [GUI: Inside Lab Canvas]

UI & System Placement: *Lab Workbench Top Toolbar > **Traffic Heatmap** button (#pnq-btn-heatmap) & Context Menu*

Operator Workflow (Step-by-Step How-To):

- **Step 1:** In the top workbench toolbar, click the pulse icon button labeled **Traffic Heatmap**.
- **Step 2:** Observe canvas links transition to dynamic color codes: Green (idle/normal <1 Mbps), Yellow (moderate 1-10 Mbps), Red (saturated >10 Mbps).
- **Step 3:** To sniff packets on a specific interface, right-click any device node and select **Live Capture (Wireshark)**.
- **Step 4:** The capture drawer opens showing live packet decode, or click **Download PCAP** for local Wireshark analysis.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Heatmap engine polls GET `/azam-ops/api/link-stats` every 3 seconds.
- **Step 2:** Backend reads virtual TAP counters from `/sys/class/net/vnet*/statistics/rx_bytes` and calculates delta rate (bytes/sec).
- **Step 3:** Front-end matches virtual interfaces to SVG link paths, updating stroke color dynamically.
- **Step 4:** For packet capture, host spawns `tcpdump -i vnetX_Y -U -w -` and streams raw PCAP bytes over HTTP.

16. AI Lab Copilot & HTML5 OS-Level Desktop Notifications [GUI: Inside Lab Canvas]

UI & System Placement: *Lab Workbench > Floating Action Widget (Bottom-Right) & OS Desktop Notification Service*

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Click the floating **AI Assistant** widget at the bottom-right corner of the canvas.
- **Step 2:** Type a query: 'Why isn't BGP EVPN establishing between Leaf1 and Leaf2?' or 'Generate Arista MLAG config'.
- **Step 3:** The AI Copilot analyzes the active topology and streams back verified configuration snippets.
- **Step 4:** If you minimize your browser during long operations, HTML5 Desktop Push Notifications will alert your operating system upon completion.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Client serializes current topology structure (nodes, images, links) and queries POST `/api/azam/ai-copilot`.
- **Step 2:** AI Copilot engine analyzes network architecture against vendor syntax guides (Cisco, Arista, Juniper).
- **Step 3:** Desktop notification helper `azamNotifyDesktop()` invokes the browser HTML5 Notification API.
- **Step 4:** Pushes native desktop OS toast notifications when Anti-Bootstorm finishes or system health warnings occur.

17. Multi-Node Config Diff & Rollback Engine [GUI: Inside Lab Canvas]

UI & System Placement: *Lab Workbench > Canvas Management Menu > **Config Diff Engine***

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Inside the lab, click **Management Tools** in the canvas sidebar > **Config Diff Engine**.
- **Step 2:** Select nodes to compare or choose between two historical configuration snapshots.
- **Step 3:** The split-screen diff viewer highlights added lines in green, removed lines in red, and modified syntax in yellow.
- **Step 4:** Click **Rollback Node Config** to push previous baseline syntax back to the device.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Queries `/opt/unetlab/tmp/{lab_id}/{node_id}/startup-config` and running snapshots.

- **Step 2:** Python difflib compares normalized configuration text, ignoring non-functional whitespace.
- **Step 3:** Renders unified diff payload and returns structured JSON to client-side CodeMirror diff editor.
- **Step 4:** On rollback, writes chosen config directly to node flash storage via unl wrapper.

18. Automated Lab Exam Grader [\[GUI: Inside Lab Canvas\]](#)

UI & System Placement: *Lab Workbench Top Bar > Lab Actions > Run Exam Evaluation*

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Click **Lab Actions** in the canvas top menu > **Run Lab Assessment / Exam Grader**.
- **Step 2:** Select test rubric (e.g. CCNA Routing, BGP Multi-Homing, OSPF Area Verification).
- **Step 3:** Click **Execute Validation Tests**.
- **Step 4:** View real-time checklist: green ticks for passed objectives, red marks for failed requirements, and final score percentage.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Backend orchestrator initiates automated Telnet/SSH probes to target node console ports.
- **Step 2:** Executes verification commands (e.g. show ip route, show ip bgp summary).
- **Step 3:** Regex parsing engine validates prefix reachability, next-hop IP, and protocol neighbor states.
- **Step 4:** Generates scorecard and saves PDF/JSON certificate in /opt/unetlab/labs/{lab}/results/.

19. Ping Mesh & Traffic Generator Engine [\[GUI: Inside Lab Canvas\]](#)

UI & System Placement: *Lab Workbench Top Bar > Lab Testing > Ping Mesh & Traffic*

Operator Workflow (Step-by-Step How-To):

- **Step 1:** In the canvas menu, select **Lab Testing > Ping Mesh & Traffic Generator**.
- **Step 2:** Select participating nodes or choose 'Full Mesh All Running Nodes'.
- **Step 3:** Set packet size (64 to 9000 bytes) and rate (pps).
- **Step 4:** Click **Start Mesh Test** to view latency matrix heatmap, packet loss, and jitter.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Orchestrator dispatches lightweight ICMP/UDP echo requests across all node bridge endpoints.
- **Step 2:** Collects round-trip time (RTT) telemetry and calculates minimum, average, and maximum latency.
- **Step 3:** Renders interactive matrix heatmap in the canvas interface.
- **Step 4:** Detects path MTU blackholes if packets above 1500 bytes drop unexpectedly.

20. High-Resolution Lab Diagram Exporter [\[GUI: Inside Lab Canvas\]](#)

UI & System Placement: *Lab Workbench Top Bar > Export > Export Topology Diagram*

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Inside the canvas, click **Export** in the top-right toolbar > **Export Topology Vector/PNG**.
- **Step 2:** Choose output format: **Scalable Vector Graphics (SVG)** or **High-Res PNG (300 DPI)**.
- **Step 3:** Toggle options: Include IP labels, Include interface names, Watermark with company logo.
- **Step 4:** Click **Download Image File** for presentation-ready architecture diagrams.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** JavaScript reads SVG DOM elements representing nodes, custom shapes, text labels, and link paths.
- **Step 2:** Normalizes viewBox coordinates, converts external image icons into inline base64 data URIs.
- **Step 3:** Applies CSS styling and embeds vector fonts directly into the standalone SVG document.
- **Step 4:** Triggers instant browser download without round-tripping to server, preserving client privacy.

21. Interactive Canvas Accelerators & Minimap [\[GUI: Inside Lab Canvas\]](#)

UI & System Placement: *Lab Workbench Canvas > Bottom-Left **Minimap** & Keyboard Accelerators*

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Locate the interactive **Minimap Overview** at the bottom-left of the canvas.
- **Step 2:** Drag the viewport indicator in the minimap to pan smoothly across large topologies with 50+ nodes.
- **Step 3:** Select multiple nodes and press **Shift + S** to snap to grid or **Shift + A** for automatic alignment.
- **Step 4:** Use mouse wheel for continuous zoom from 25% overview to 200% detail view.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Canvas rendering engine creates a secondary hardware-accelerated HTML5 canvas element for the minimap.
- **Step 2:** Listens to canvas pan/zoom transform matrices and recalculates viewport bounding box in real-time.
- **Step 3:** Snap-to-grid algorithm calculates modulo-20 coordinate snapping for node drag events.
- **Step 4:** Persists updated coordinates directly to the underlying .unl XML file upon mouse release.

5. Background Daemons & Kernel Infrastructure

These core features operate autonomously beneath the graphical interface on both Master and Satellite nodes. They maintain hypervisor stability, self-heal system services, enable hardware-accelerated networking, and perform scheduled system maintenance.

22. 24/7 Self-Healing System Watchdog Daemon [Background Daemon Plane (Master & Satellite)]

UI & System Placement: *Linux Systemd Service > azam-watchdog.service (Autonomous 24/7)*

Operator Workflow (Step-by-Step How-To):

- **Step 1:** The watchdog runs autonomously as a Linux systemd service: `azam-watchdog.service` on both Master and Satellite nodes.
- **Step 2:** To inspect live status, open an SSH terminal and run: `systemctl status azam-watchdog`.
- **Step 3:** To view event logs: `tail -f /var/log/azambasha-watchdog.log`.
- **Step 4:** No manual interaction is needed; the service auto-starts on boot and runs continuously.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Daemon wakes up every 60 seconds and evaluates host health metrics.
- **Step 2:** Inspects essential processes; if any core service is down, triggers automated restart.
- **Step 3:** Monitors available system RAM; if free RAM drops below 5%, identifies hung QEMU processes and alerts.
- **Step 4:** Cleans up zombie processes and stale temporary lock files in `/tmp` to prevent hypervisor starvation.

23. MySQL Socket & Credential Auto-Healer [Background Daemon Plane (Master Only)]

UI & System Placement: *Systemd Hook & Cron > /usr/local/bin/azambasha-fix-web-credentials*

Operator Workflow (Step-by-Step How-To):

- **Step 1:** The healer runs automatically during boot, hourly via cron, and upon admin request on the Master Node.
- **Step 2:** To trigger manual repair from CLI, execute: `azambasha-fix-web-credentials`.
- **Step 3:** The script inspects database user passwords, table permissions, and socket symlinks.
- **Step 4:** Confirms with output message: `[OK] Web credentials and MySQL socket successfully verified.`

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Checks if MariaDB unix socket exists at `/var/run/mysqld/mysqld.sock`; if missing, creates symlink.
- **Step 2:** Connects to database and verifies pnetlab user credentials match `/opt/unetlab/html/includes/config.php`.
- **Step 3:** Synchronizes Guacamole database credentials in `/etc/guacamole/guacamole.properties`.
- **Step 4:** Flushes MariaDB privileges (`FLUSH PRIVILEGES;`) and verifies read/write integrity.

24. Soft-RoCE RXE & MTU 9000 Network Accelerator [Kernel Infrastructure Plane (Master & Satellite)]

UI & System Placement: *Linux Kernel Module & Network Configuration (/etc/modprobe.d/, udev)*

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Activated on both Master and Satellite nodes during setup via `azambasha-os-prerequisites.sh`.
- **Step 2:** To inspect RDMA status, run: `rdma link or ibv_devices`.
- **Step 3:** To verify Jumbo Frame MTU across bridge interfaces: `ip link show | grep mtu`.
- **Step 4:** All virtual bridges (pnet0 through pnet9) operate with MTU 9000 for high-throughput packet emulation.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Loads Linux kernel modules `rdma_rxe` and `ib_core` into kernel memory.
- **Step 2:** Binds software RoCE (RDMA over Converged Ethernet) devices to physical NIC interfaces.
- **Step 3:** Configures udev rules and `/etc/network/interfaces` setting bridge MTU to 9000.
- **Step 4:** Eliminates CPU packet fragmentation overhead for high-speed emulated links (e.g. 10G/40G data center fabrics).

25. Scheduled Maintenance & Storage TRIM Engine [\[Background Daemon Plane \(Master & Satellite\)\]](#)

UI & System Placement: *System Cron > /etc/cron.d/azambasha-maintenance (Nightly 03:00 UTC)*

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Scheduled automatically via cron on Master and Satellite nodes: /etc/cron.d/azambasha-maintenance.
- **Step 2:** Executes nightly at 03:00 UTC without disrupting running user labs.
- **Step 3:** To run on demand from CLI: azambasha-heavy-node-optimizer.sh --maintenance.
- **Step 4:** Check maintenance log at: /var/log/azambasha-maintenance.log.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Executes fstrim -av to issue TRIM commands to underlying SSD/NVMe storage, recovering deleted blocks.
- **Step 2:** Clears Linux dentry and inode pagecache buffers via sysctl vm.drop_caches=3 if system is idle.
- **Step 3:** Truncates log files in /opt/unetlab/data/Logs/ older than 14 days and compresses historical archives.
- **Step 4:** Empties orphaned sockets in /opt/unetlab/tmp/ left behind by abnormally terminated nodes.

6. Enterprise CLI Command-Line Reference & Distribution

All twenty-six Azam-Pnet enterprise utilities are symlinked globally in /usr/local/bin/ across both Master and Satellite nodes. This ensures that infrastructure engineers possess identical troubleshooting capabilities from SSH terminals regardless of node type.

Command / Utility	Target Feature / Scope	Typical Invocation / Options	Node Support
azam-airgap-pack	Offline Air-Gapped Bundler	azam-airgap-pack [output_dir]	Master & Satellite
azam-watchdog	24/7 Autonomous Watchdog	systemctl status azam-watchdog	Master & Satellite
azambasha-ops-api	REST API Daemon Backend	systemctl restart azam-ops-api	Master Only
azambasha-bootstorm	Anti-Bootstorm & KSM Engine	azambasha-bootstorm --lab <lab.unl>	Master & Satellite
azambasha-doctor	Diagnostic Health Suite	azambasha-doctor --full --auto-heal	Master & Satellite
azambasha-fix-web-credentials	MySQL & Guac Socket Fixer	azambasha-fix-web-credentials --fix-guac	Master Only
azambasha-heavy-node-optimizer	QCOW2 Disk & RAM Optimizer	azambasha-heavy-node-optimizer --shrink	Master & Satellite
azambasha-cloud-backup	Local & Cloud Backup Engine	azambasha-cloud-backup --local --cloud s3	Master & Satellite
azambasha-cloud-transit	Cloud Transit Overlay Bridge	azambasha-cloud-transit --tailscale --connect	Master & Satellite
azambasha-install-azam-features	Symlink & Package Installer	azambasha-install-azam-features.sh [--satellite]	Master & Satellite
azambasha-fix-network-boot	Bridge & TAP Network Boot Fix	azambasha-fix-network-boot.sh	Master & Satellite
azambasha-fix-export-and-apt	Ubuntu Package Repository Fix	azambasha-fix-export-and-apt.sh	Master & Satellite

7. Future VM Automated Provisioning & Verification Guide

To guarantee 100% feature alignment on every future virtual machine, the installer scripts in the Azam-Pnet repository have been standardized into two automated workflows:

Future Master VM Provisioning: 1. Boot a fresh Ubuntu 20.04/22.04/24.04/26.04 server. 2. Clone the repository to /opt/azambasha. 3. Execute bash /opt/azambasha/install.sh. 4. The installer automatically provisions all web assets, MariaDB credentials, Apache/Nginx routing, compiles the REST API daemon, links all 26 CLI tools, and verifies health via Test 6.	Future Satellite VM Provisioning: 1. Boot a fresh Ubuntu server with KVM virtualization enabled. 2. Clone the repository to /opt/azambasha. 3. Execute bash /opt/azambasha/install-satellite.sh. 4. In Step 9, the script automatically triggers azambasha-install-azam-features.sh --satellite, deploying all 26 CLI symlinks, watchdog daemon, KSM tuning, and TRIM cron in headless mode.
--	--

Documentation Summary & Compliance Note:

This manual encompasses all 25 enterprise features and 9 advanced platform enhancements integrated into the Azam-Pnet platform. All API endpoints, UI placements, and background scripts adhere to the platform's non-destructive design architecture. Stale websockets, hypervisor memory thresholds, and disk structures are guarded by automated checks to guarantee continuous lab uptime across both Master and Satellite compute nodes.